

실시간 얼굴 인식 기술을 이 용한 가상 마스크 필터 기능 구현 프로젝트

19105332 컴퓨터공학 조동영

- 1. 프로젝트 개요
- **기획 의도:** 기존의 정적인 필터를 넘어, 사용자의 얼굴과 손동작을 실시간 인식하여 인터랙션(상호작용)이 가능한 증강현실 인터페이스 구현.

- 2.시스템 구성도:
- 흐름: Webcam Stream -> MediaPipe FaceMesh/Hands
Landmark Extraction -> Filter Logic Processing -> Alpha
Blending Rendering -> Output

3. 핵심 기술 구현:

- **동적 스케일링 (Dynamic Scaling):** "사용자 얼굴의 미간과 뺨 랜드마크를 활용하여, 카메라와의 거리에 관계없이 필터가 사용자 얼굴에 정밀하게 피팅되도록 구현함."
- **독립적 필터 제어:** "각 필터별로 좌표 처리 모듈을 분리하여, 여러 필터를 동시에 활성화해도 좌표 간섭이 없도록 설계함."
- **이벤트 처리 (Event Handling):** "OpenCV mouseCallback을 활용한 GUI 토글 방식 적용."

4. 주요 코드 섹션

- **overlay_png 함수:** "투명 배경(PNG) 합성을 위한 알파 블렌딩 알고리즘 구현부."

```
# [얼굴 전체 가면 필터]
if self.filters['face']:
    left_x = int(face_landmarks.landmark[234].x * w)
    right_x = int(face_landmarks.landmark[454].x * w)
    face_width = right_x - left_x

    center_x = int(face_landmarks.landmark[168].x * w)
    center_y = int(face_landmarks.landmark[168].y * h)

    if self.img_face is not None and len(self.img_face.shape) == 3 and self.img_face.shape[
        2] == 4:
        img_h, img_w = self.img_face.shape[:2]
        img_ratio = img_h / img_w
        scale_factor = 4.5
        mask_w = int(face_width * scale_factor)
        mask_h = int(mask_w * img_ratio)
        resized_mask = cv2.resize(self.img_face, (mask_w, mask_h))
        offset_y = int(mask_h * 0.45)
        frame = self.overlay_png(frame, resized_mask, center_x - mask_w // 2,
                                center_y - offset_y)
```

- 사용자 얼굴의 미간과 양쪽 뺨 랜드마크를 활용하여, 카메라 거리 변화에 실시간으로 대응하는 동적 스케일링(Dynamic Scaling) 시스템을 구현함

```

# 🐼 [사륜안 개안 로직] 🐼
if self.filters['sharingan']:
    # 홍채 중심점 468, 473 번 사용!
    lx_in, ly_in = int(face_landmarks.landmark[133].x * w), int(
        face_landmarks.landmark[133].y * h)
    lx_out, ly_out = int(face_landmarks.landmark[33].x * w), int(
        face_landmarks.landmark[33].y * h)
    l_iris_x, l_iris_y = int(face_landmarks.landmark[468].x * w), int(
        face_landmarks.landmark[468].y * h)
    l_eye_width = int(np.sqrt((lx_in - lx_out) ** 2 + (ly_in - ly_out) ** 2))

    rx_in, ry_in = int(face_landmarks.landmark[362].x * w), int(
        face_landmarks.landmark[362].y * h)
    rx_out, ry_out = int(face_landmarks.landmark[263].x * w), int(
        face_landmarks.landmark[263].y * h)
    r_iris_x, r_iris_y = int(face_landmarks.landmark[473].x * w), int(
        face_landmarks.landmark[473].y * h)
    r_eye_width = int(np.sqrt((rx_out - rx_in) ** 2 + (ry_out - ry_in) ** 2))

    if self.img_sharingan is not None and len(self.img_sharingan.shape) == 3 and W
        self.img_sharingan.shape[2] == 4:
            scale = 0.4
            l_size = max(1, int(l_eye_width * scale))
            r_size = max(1, int(r_eye_width * scale))

            resized_l_eye = cv2.resize(self.img_sharingan, (l_size, l_size))
            resized_r_eye = cv2.resize(self.img_sharingan, (r_size, r_size))

            frame = self.overlay_png(frame, resized_l_eye, l_iris_x - l_size // 2,
                                     l_iris_y - l_size // 2)
            frame = self.overlay_png(frame, resized_r_eye, r_iris_x - r_size // 2,
                                     r_iris_y - r_size // 2)

```

MediaPipe Face Mesh의 refine_landmarks=True 설정을 통해 홍채(Iris) 중심 좌표를 정밀 추적함. 이를 통해 사용자의 시선 이동에 따라 필터가 실시간으로 동기화되어 렌더링됨."

```
def overlay_png(self, bg_img, img_to_overlay_t, x, y):
    if img_to_overlay_t is None or len(img_to_overlay_t.shape) < 3 or img_to_overlay_t.shape[2] < 4:
        return bg_img

    bg_img = bg_img.copy()
    h, w, _ = img_to_overlay_t.shape
    bg_h, bg_w, _ = bg_img.shape

    y1, y2 = max(0, y), min(bg_h, y + h)
    x1, x2 = max(0, x), min(bg_w, x + w)
    y1_o, y2_o = max(0, -y), min(h, bg_h - y)
    x1_o, x2_o = max(0, -x), min(w, bg_w - x)

    if y1 >= y2 or x1 >= x2 or y1_o >= y2_o or x1_o >= x2_o:
        return bg_img

    alpha = img_to_overlay_t[y1_o:y2_o, x1_o:x2_o, 3] / 255.0
    alpha_inv = 1.0 - alpha

    for c in range(3):
        bg_img[y1:y2, x1:x2, c] = (alpha * img_to_overlay_t[y1_o:y2_o, x1_o:x2_o, c] +
                                   alpha_inv * bg_img[y1:y2, x1:x2, c])
    return bg_img
```

이미지의 RGBA 및 Alpha 채널을 활용한 알파 블렌딩 알고리즘을 구현하고, 얼굴을 실시간으로 추적하여 움직임에 맞게 가상 마스크를 자동 정합하는 AR 환경을 구현함.

5.구현 결과 및 시연:





Face

Glasses

Eye

Nose

Mouth

Hand



Face

Glasses

Eye

Nose

Mouth

Hand



Face

Glasses

Eye

Nose

Mouth

Hand



Face

Glasses

Eye

Nose

Mouth

Hand



Face

Glasses

Eye

Nose

Mouth

Hand

결과 분석: "다양한 얼굴 각도와 거리에서도 안정적으로 필터가 적용됨을 확인."

- 6.결론 및 트러블슈팅
- **경험:** "초기 MediaPipe 라이브러리 연동 시 경로 환경 문제 발생했으나, 가상환경(venv) 재설정 및 No-Cache 강제 설치를 통해 안정적인 빌드 환경을 구축함.
- 특히 이미지 합성 시 발생하는 좌표 오프셋(Offset) 문제는 행렬 변환 및 거리 기반 동적 스케일링 보정을 통해 해결하였음."